

2110531 - Data Science and Data Engineering Tools

Data Lakehouse

Asst. Prof. Natawut Nupairoj, Ph.D.

Department of Computer Engineering, Chulalongkorn University

natawut.n@chula.ac.th

CHULA ENGINEERING
Foundation toward Innovation

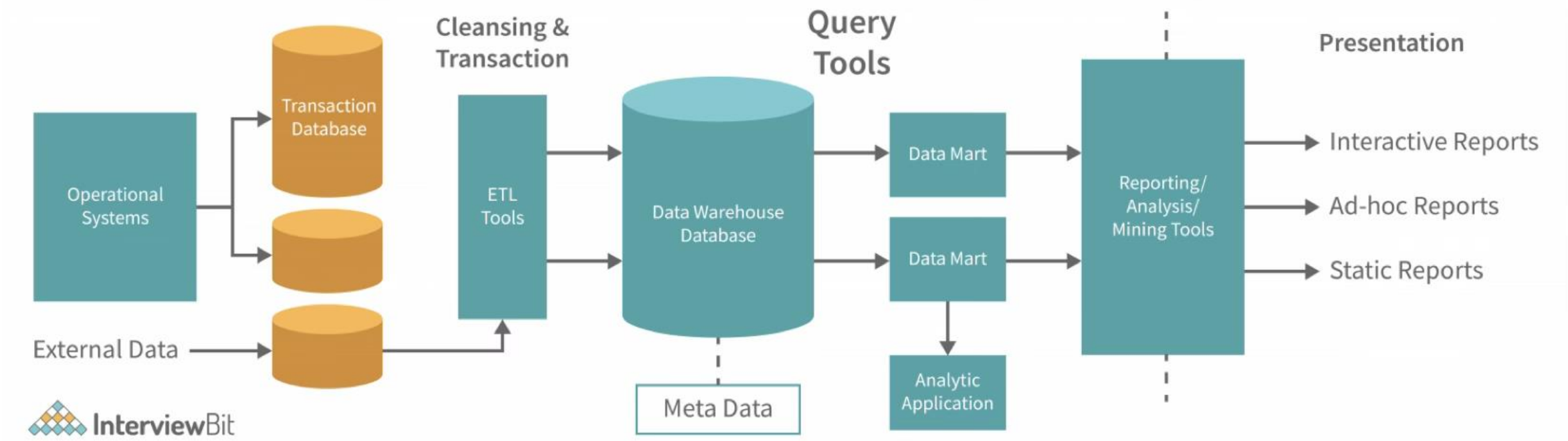
Business needs different kinds of storage

- Operational systems support day-to-day transactions
- Data Warehouses support curated reporting
- Data Lakes retain diverse data at scale
- Data Lakehouses support governed analytics over diverse data

Data Warehouse starts with known questions

- Data are modeled before routine reporting
- Definitions and transformations are controlled
- Well suited to stable metrics and dashboards
- Strong consistency for known analytical use cases
- Less flexible for raw images, signals, logs, and rapidly changing data

Data Platform: Data Warehouse

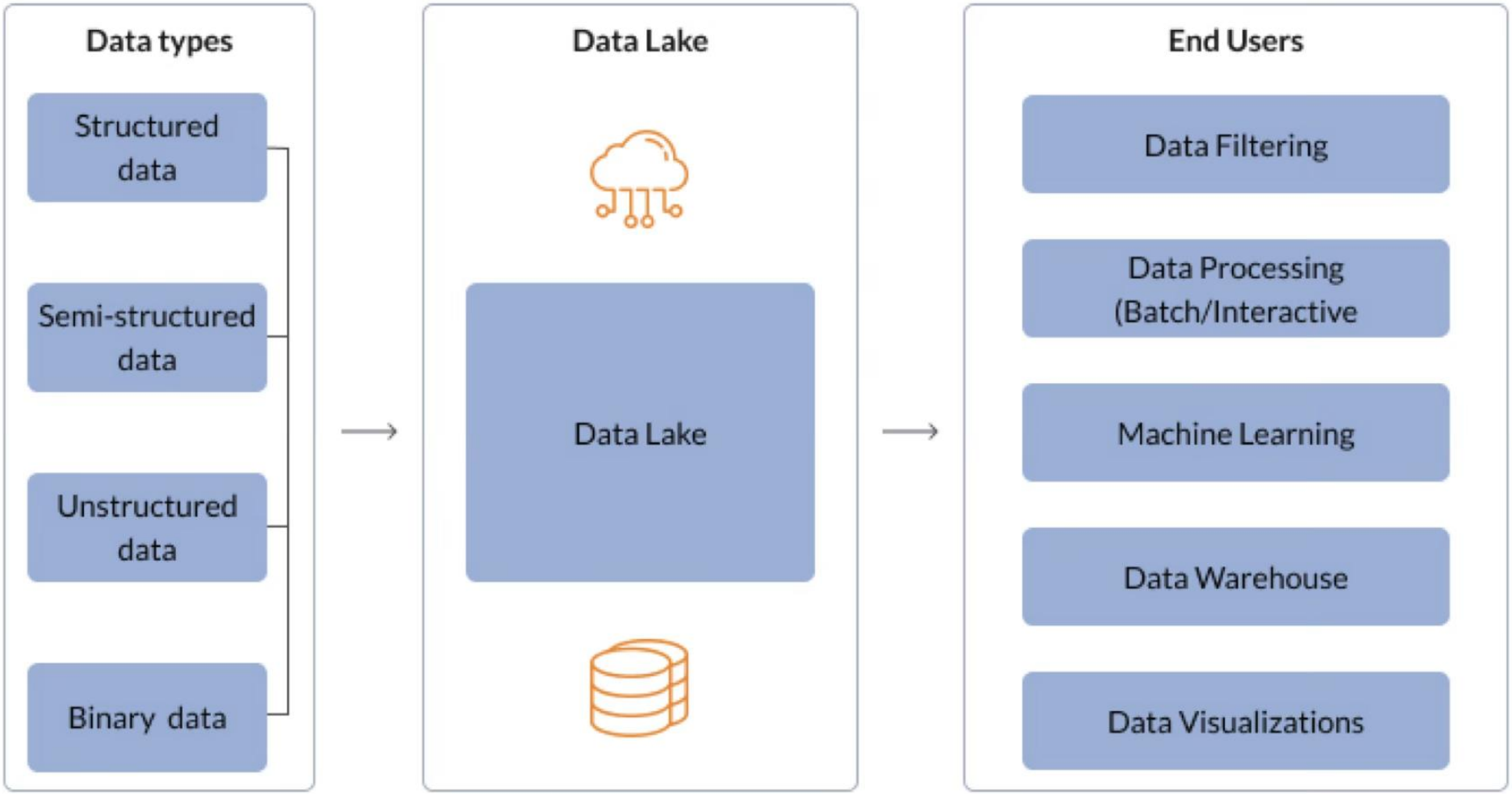


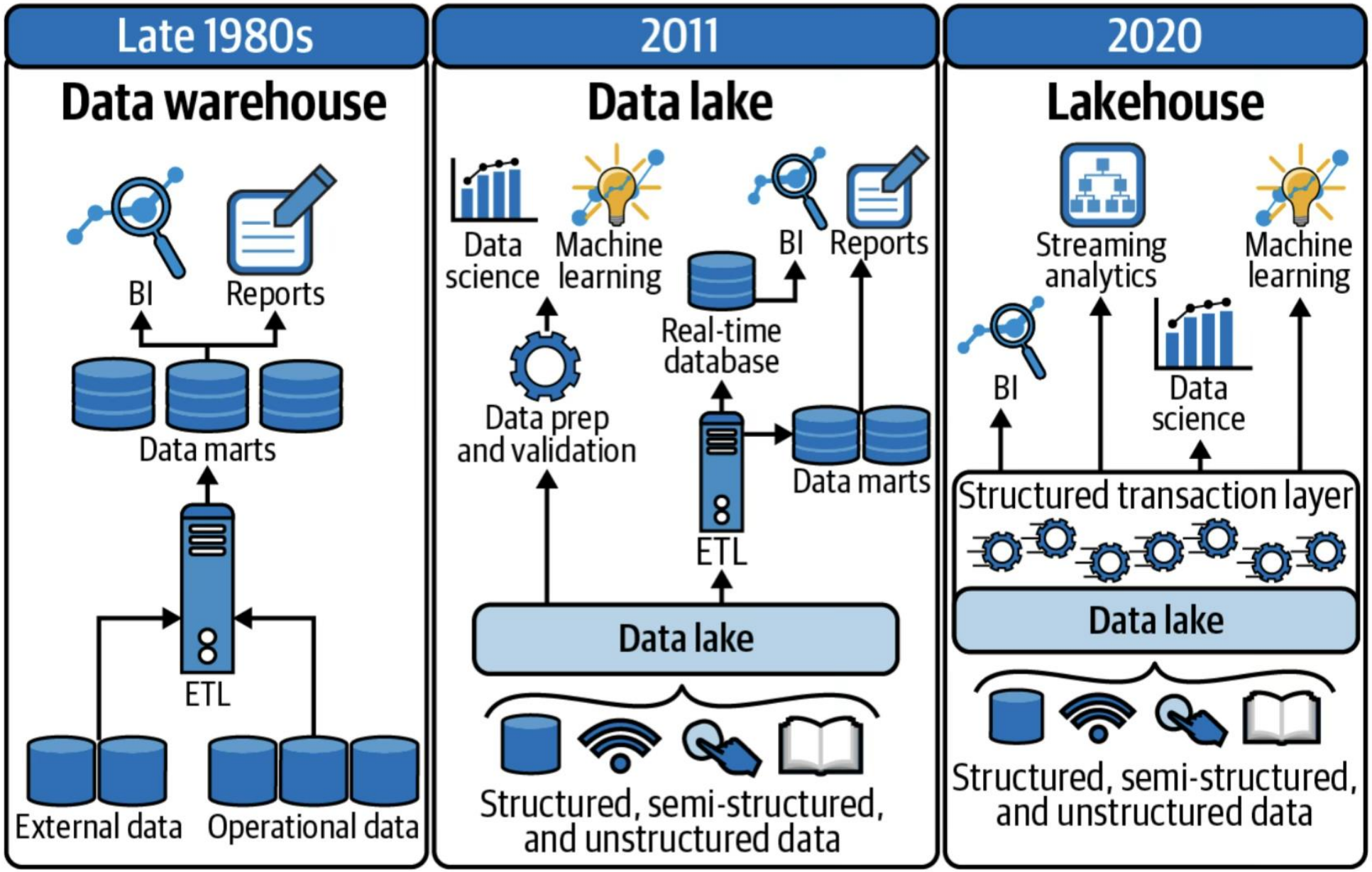
Source: “Data Warehouse Architecture – Detailed Explanation”, https://www.interviewbit.com/blog/data-warehouse-architecture/

Data Lake retains diverse source data

- Structured, semi-structured, and unstructured data
- Large-scale and relatively flexible storage
- Original data can be retained for future use
- Useful for exploration, research, and machine learning
- Weak governance can turn a lake into an unusable collection

Data Platform: Data Lake





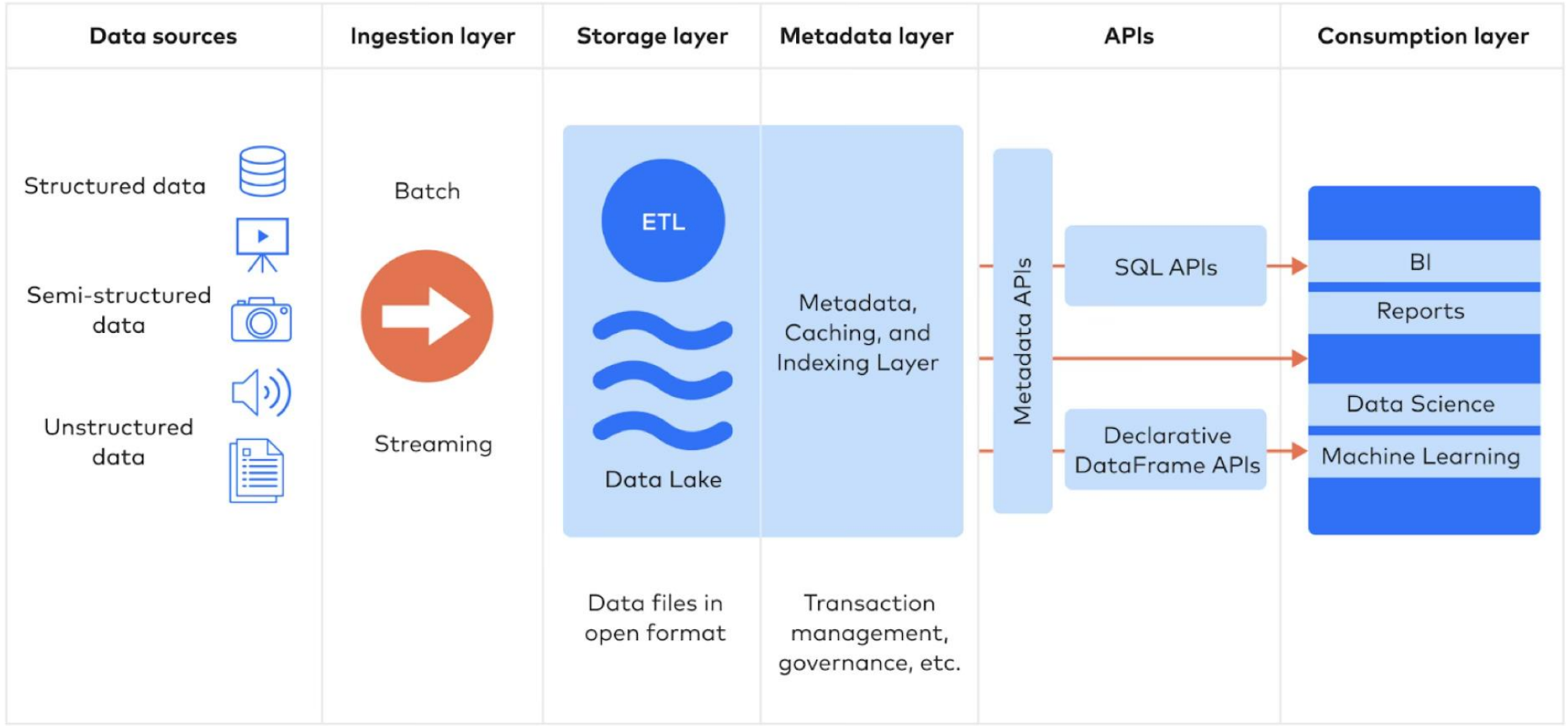
Source: Alice LaPlante, “The Modern Cloud Data Platform”

CHULA ENGINEERING
Foundation toward Innovation

Data Lakehouse combines flexibility and control

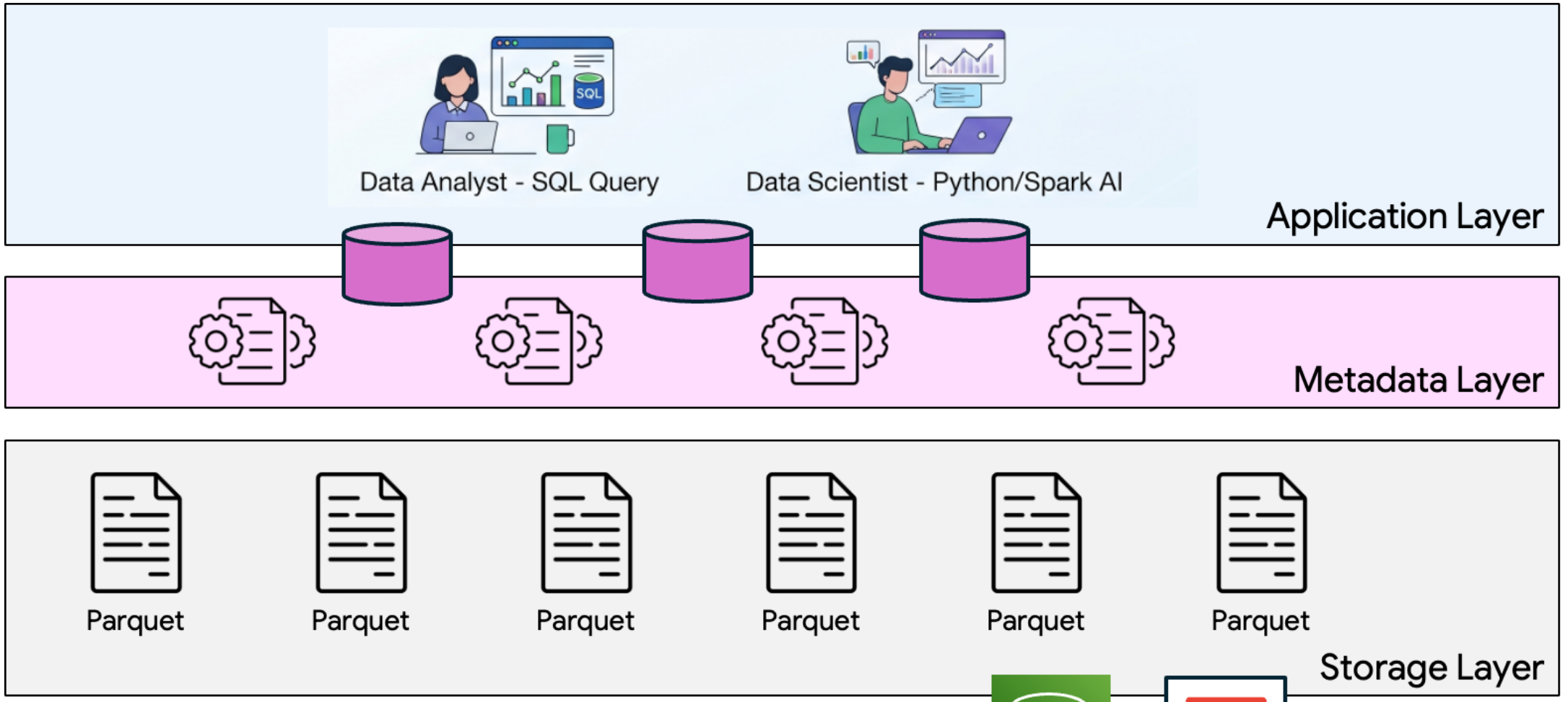
- Diverse data remain available at scale
- Tables and metadata make datasets discoverable
- Data-quality rules support trustworthy downstream use
- Batch and streaming data can converge
- Analytics and AI can use governed, traceable datasets

Data Lakehouse Architecture



Source: https://www.fivetran.com/blog/what-is-a-data-lakehouse

Data Lakehouse Architecture



Data Lakehouse Architecture

- Storage Layer:
  - Object storage (similar to data lake) for cost effectiveness (AWS S3, Google Cloud Storage, MinIO, HDFS)
  - Support structured and unstructured
  - Use Open File Formats e.g. Parquet, ORC, etc.
- Metadata Layer:
  - Open Table Formats e.g. Delta Lake, Apache Iceberg, Apache Hudi
  - Handle schema evolution, time-travel, and data access control
- Compute & Application Layer:
  - Data clients accessing via direct access (e.g. Python, Spark, etc.) and query engine (e.g. Trino, Dremio, DuckDB)

Table-Oriented Data Formats

- Typical data analytics are done on “table” of data
- For big data, a table may not be fit in a single file
- Multiple files can be processed in parallel effectively
- Row-store file format is more efficient for add/update/delete operations
- Column-store file format is more efficient for data analytic operations

Table

	Country	Product	Sales
Row 1	India	Chocolate	1000
Row 2	India	Ice-cream	2000
Row 3	Germany	Chocolate	4000
Row 4	US	Noodle	500

Row Store

Row 1	India
	Chocolate
	1000
Row 2	India
	Ice-cream
	2000
Row 3	Germany
	Chocolate
	4000
Row 4	US
	Noodle
	500

Column Store

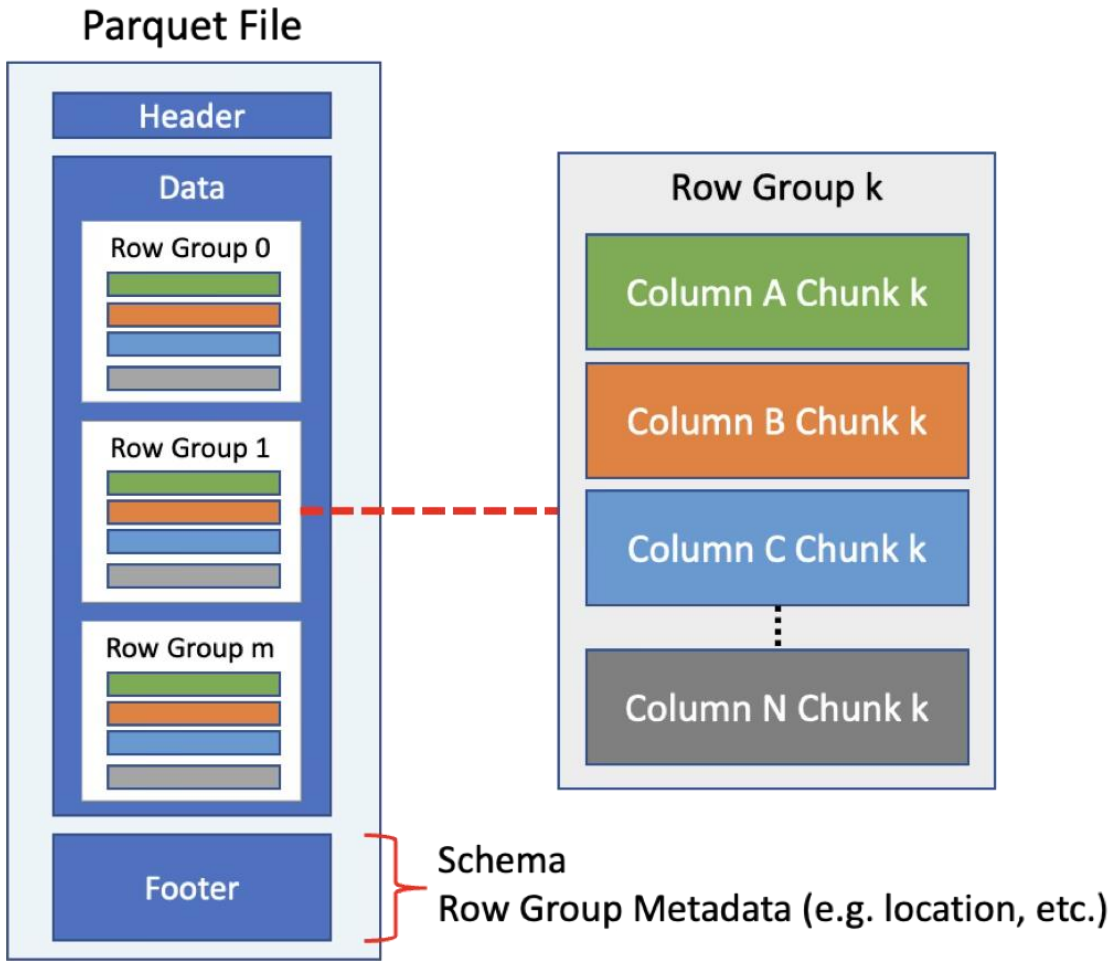
Country	India
	India
	Germany
	US
Product	Chocolate
	Ice-cream
	Chocolate
	Noodle
Sales	1000
	2000
	4000
	500

Parquet Format

- Columnar schema-oriented data store
- Analytic operations are usually column-based
- Schema is embedded in the file
- Support nested schema using record shredding and assembly algorithm (Google Dremel) to map hierarchical data into column-based data store
- Support column-wise data compression
  - More efficient as each column shares the same datatypes
  - Run length encoding, dictionary encoding, delta encoding
- Support in multiple languages, as well as, pandas, MapReduced, and spark

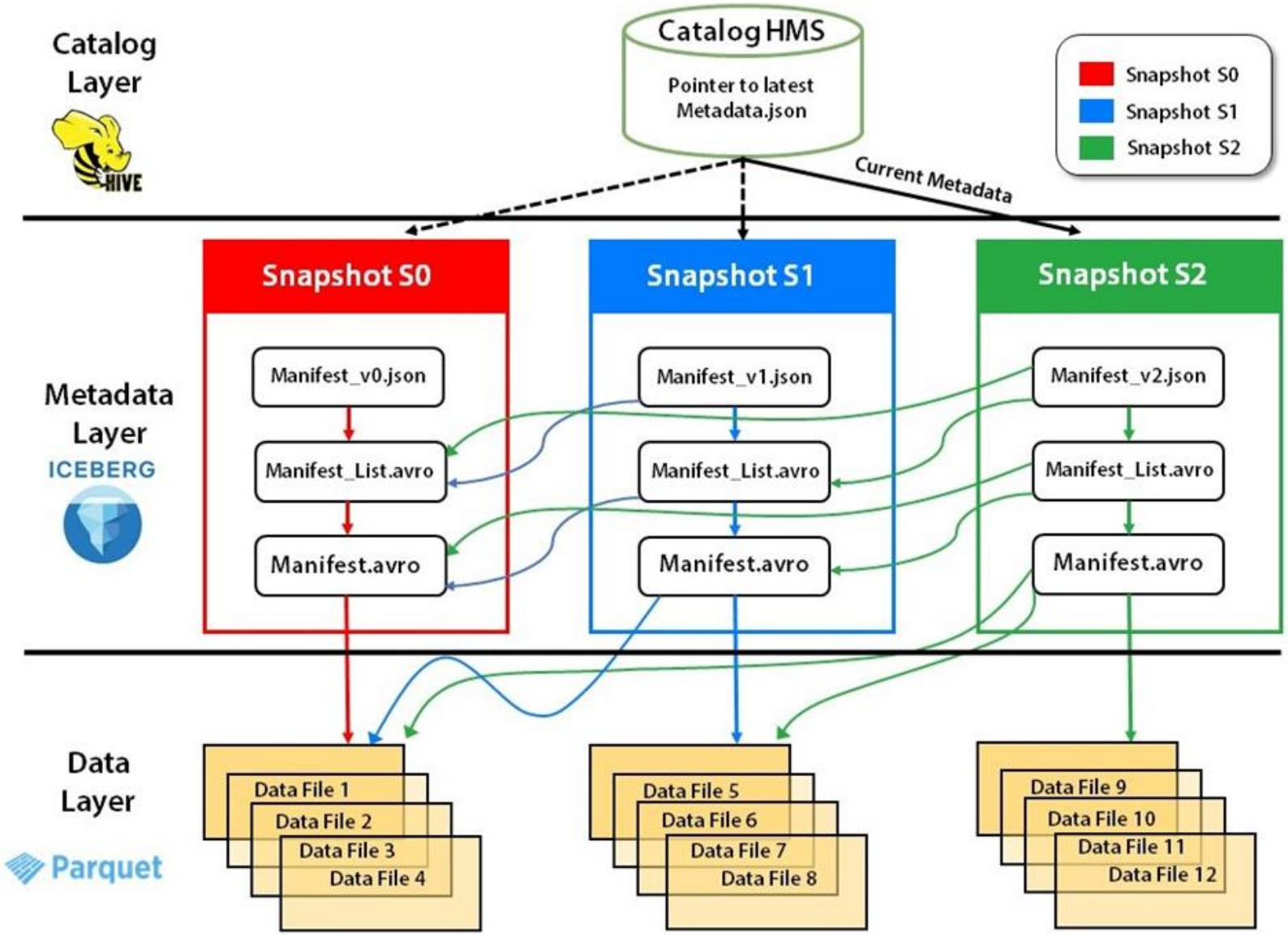
Parquet File Structure

- Header (4 bytes) ‘PAR1’ — indicate Parquet file format
- Data contains set of row groups
  - Partition data into row groups (m row groups in the example) — for MapReduce parallelization
  - Each row group contains all columns (from A to N in the example) and metadata of the group
  - Column data in the same row group is stored in a column chunk with metadata of the chunk — for I/O parallelization
- Footer contains schema and metadata of the file (e.g. location of metadata of each row group and each column in row group)



Metadata Layer: Apache Iceberg

- 3-layer design for governing how data is written, queried, and modified
- Catalog layer
  - Keep track of the current metadata pointer
  - Tell SQL engines where the latest table version is
- Metadata layer
  - Handles versioning, file management, schema history
- Data Layer
  - Immutable files in Parquet, ORC, or Avro
  - Never updates data files in-place — metadata tracks file versions instead

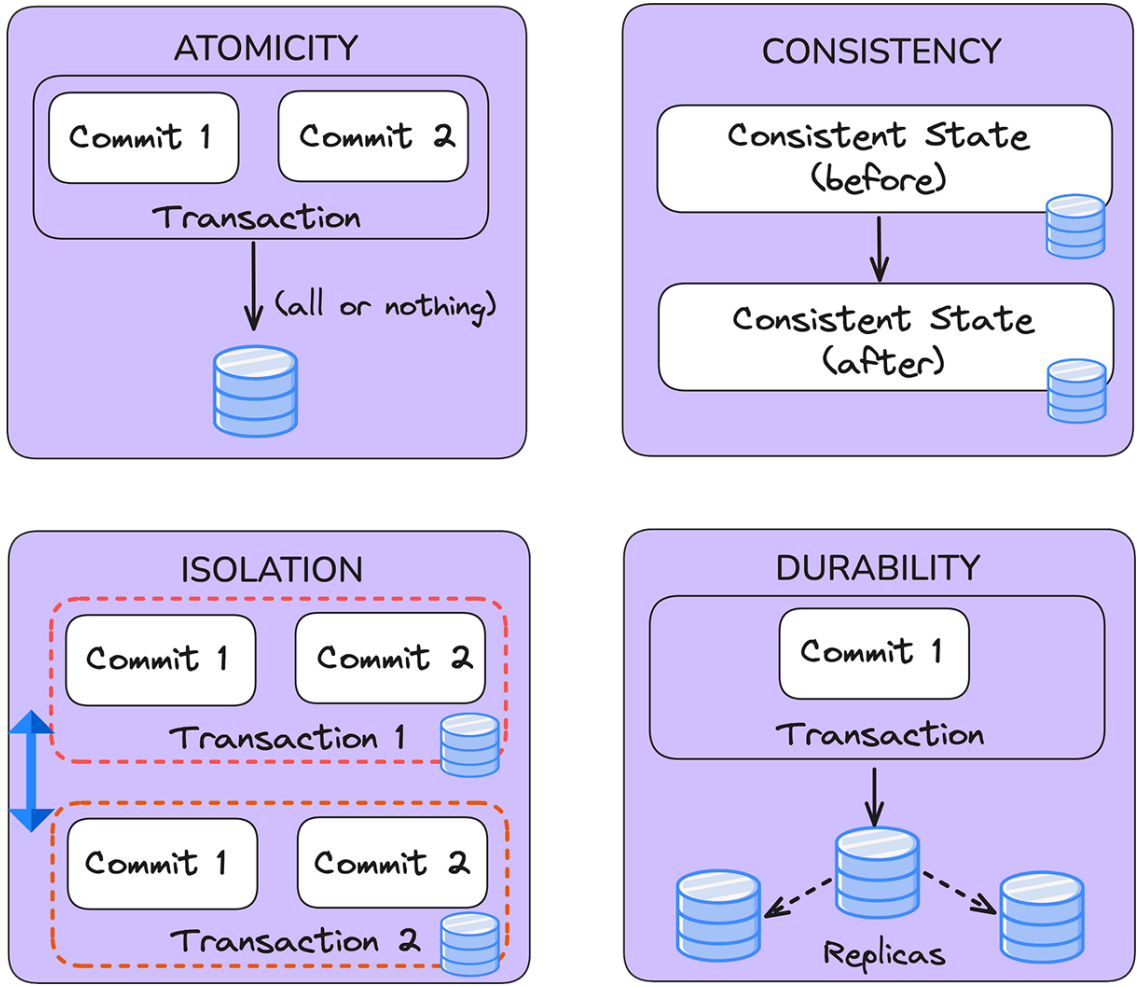


https://medium.com/@mohammad.atif1987/apache-iceberg-metadata-architecture-demystified-powering-the-modern-data-lakehouse-8b6069045ae9

Data Lakehouse Features

- ACID Transactions
- Schema Enforcement and Evolution
- Time Travel and Versioning
- Decoupled Storage and Compute
- Support for Diverse Data Types
- Unified Workloads (BI and AI/ML)
- Streaming and Batch Processing
- Centralized Governance

ACID Transactions



- Transaction - a sequence of operations executed as a single logical unit of work
- **Atomicity**: either all operations succeed, or none do (All or Nothing)
- **Consistency**: transactions respect the rules and constraints of the system, preserving its integrity
- **Isolation**: transactions operate independently of one another, even if they are executed concurrently
- **Durability**: once committed, a transaction's effects persist

Source: https://www.onehouse.ai/blog/acid-transactions-in-an-open-data-lakehouse

ACID in Data Lakehouse

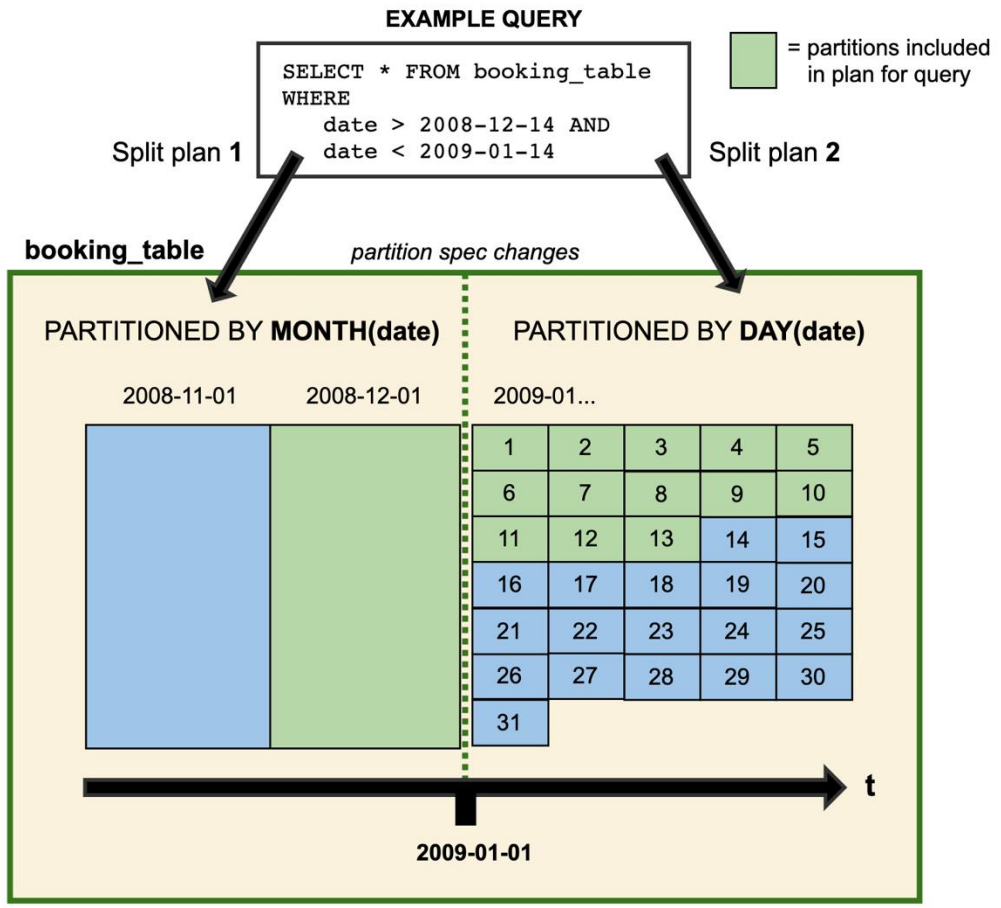
- Consistency Across Data Ingestion and Querying
  - Ensure (parallel) ingestion jobs from multiple sources are treated as atomic transaction
- Concurrent Query Isolation
  - Provide snapshot isolation, allowing each query to operate on a consistent dataset without interference from other ongoing processes e.g. query, ingestion, etc.
- Durability for Recovery and Reliability
  - Ensures that fully committed transactions persist, while partial writes or failed operations are rolled back

Schema Evolution

- Ability to modify table structures—such as adding, deleting, or renaming columns and changing data types—over time without breaking queries or rewriting entire historical datasets
- Apache Iceberg
  - Tracks column IDs rather than names or positions
  - Schema updates are metadata changes, so no data files need to be rewritten
  - Allow seamless column renames, drops, and type promotions without altering underlying data files

Partition Evolution

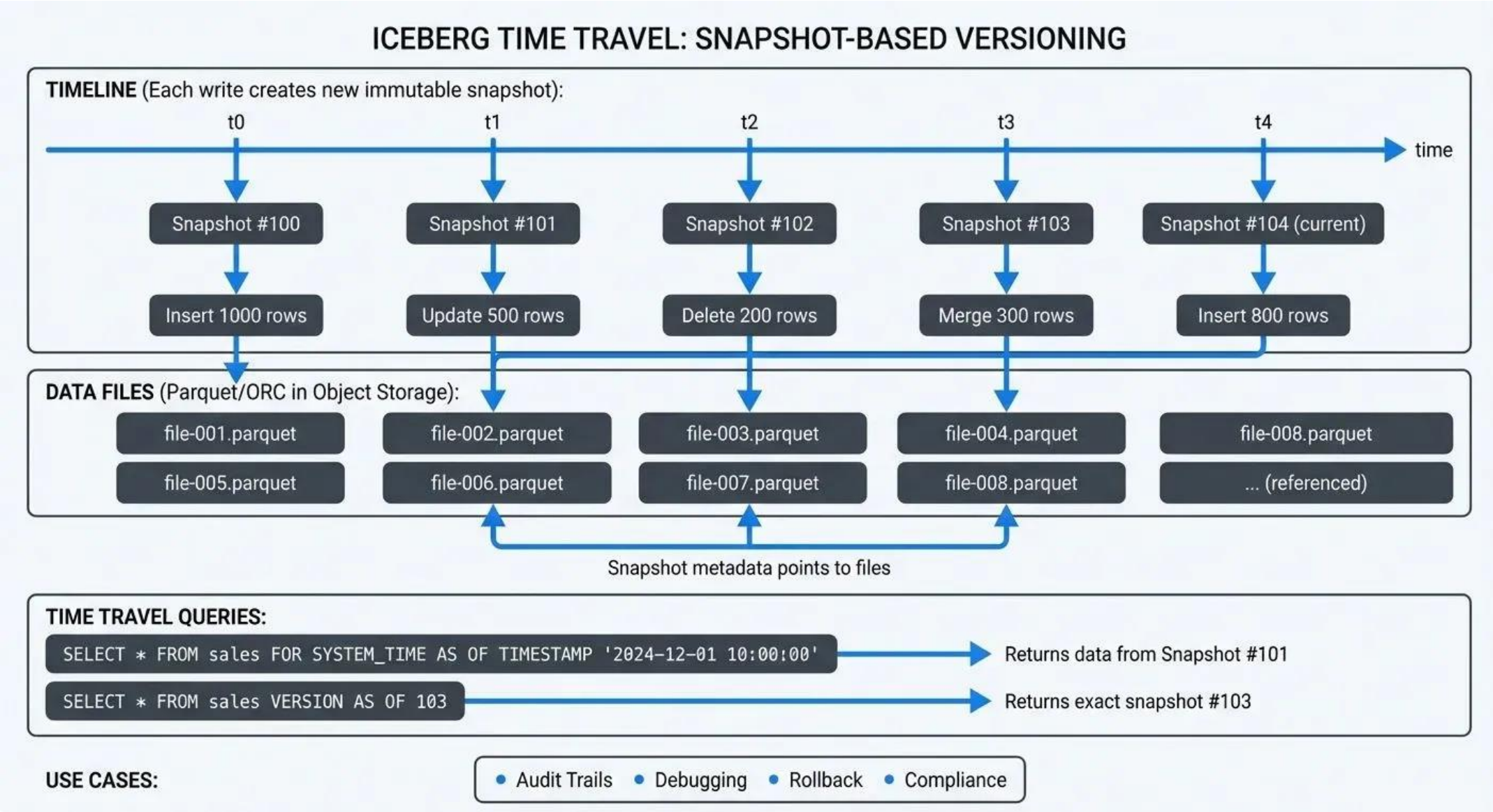
- Table partitioning splits a large logical table into smaller, independent physical pieces
- To improve query performance, manageability, and maintenance
- Partition evolution allows table's partition layout in-place through metadata operations without rewriting historical data files



CHULA ENGINEERING
Foundation toward Innovation

Snapshots

- A point-in-time, immutable record of a table's metadata and state; a complete, consistent view of the table at that moment in time
- Every write operation: insert, update, delete, or merge, creates a new immutable snapshot of the table
- Similar to Git where each commit creates an immutable reference to the repository state
- Enable atomic transactions and time travel

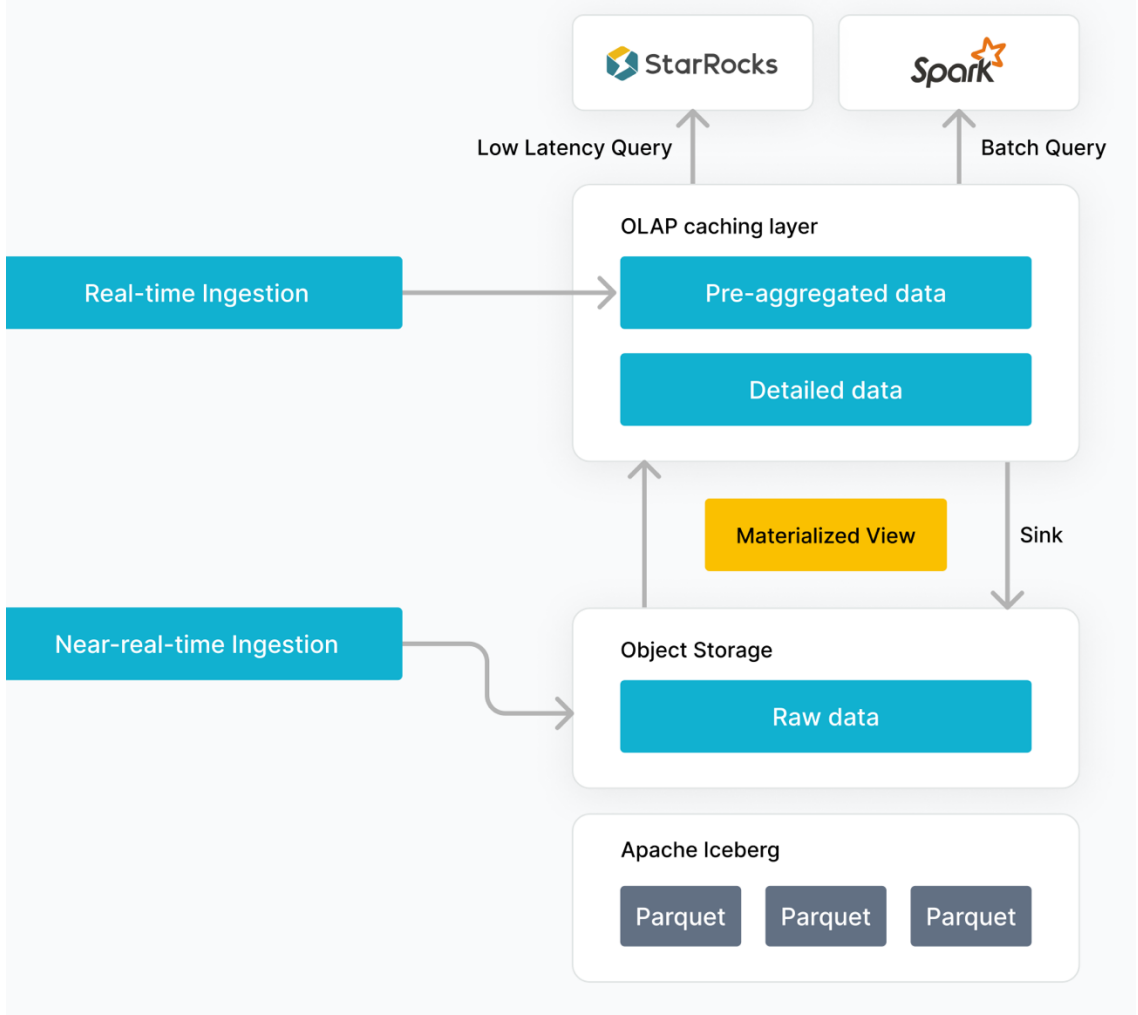


Source: https://www.conduktor.io/glossary/time-travel-with-apache-iceberg

Data Lakehouse Case Study: WeChat

- Support 1.3B+ monthly active users; some tables grow by trillions of records daily, queries scan 1B+ records
- Challenge: Separate Hadoop data lakes and warehouses created duplicate data, fragmented batch/real-time pipelines, and high operational overhead
- Need unified ingestion: Real-time queries combine hot warehouse data with cold lake data; near-real-time data lands directly in lakehouse metadata layer and is transformed using materialized views

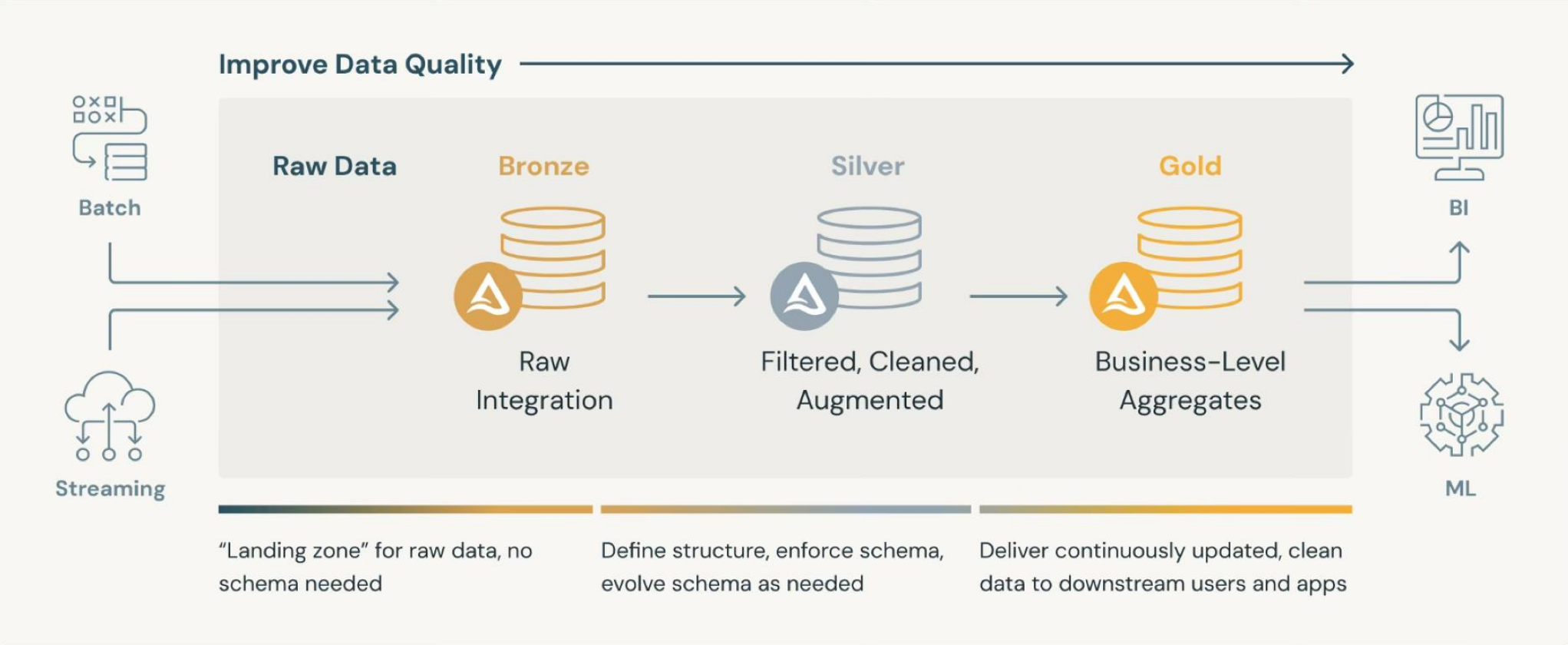
WeChat Data Lakehouse Architecture



- Tencent Cloud Object Storage
- Parquet for scalable storage
- Apache Iceberg for open table management
- Apache Spark for batch processing
- StarRocks for low-latency analytics
- Performance: Data freshness improved to seconds–10 minutes; detailed-data queries run in seconds or less, while pre-processed queries achieve sub-second latency

https://www.starrocks.io/hubfs/Case%20Study%20PDFs/WeChat%20Case%20Study？hsLang=en.pdf

Medallion Architecture



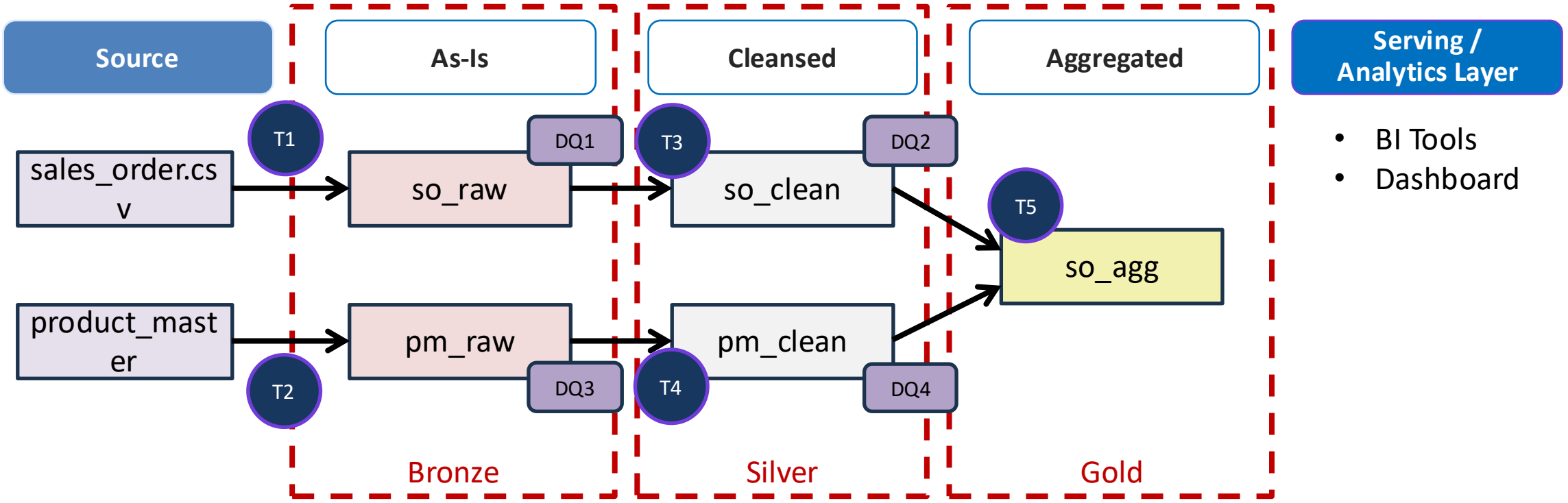
All the processing and enrichment of data from Bronze (raw data) to Silver (cleansed, filtered) to Gold (structured and ready to be used by analytics, reporting, and data science) happens within Delta Lake

CHULA ENGINEERING
Foundation toward Innovation

Health Data and Medallion Architecture layer

- Raw (Bronze): healthcare data, labs data, IoT events, notes, images, logs
- Validated (Silver): normalized units, matched identities, checked timestamps
- Curated (Gold): patient timelines, quality indicators, research cohorts
- We can have more than 3 layers
- AI datasets require purpose-specific selection and validation

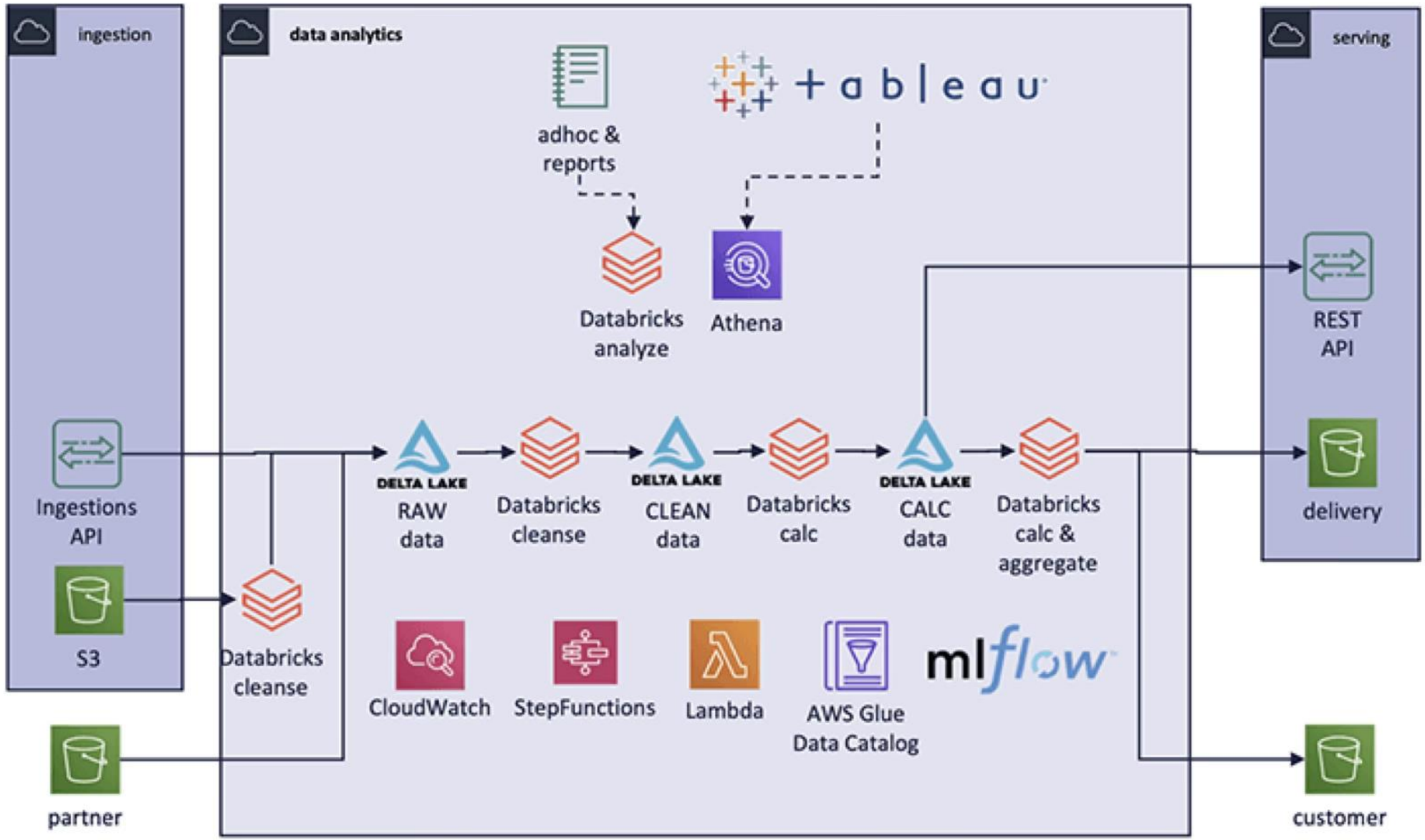
Medallion “Multi-Hop” Architecture Pipeline



Series of transformation scripts and data quality gates is incrementally and progressively improving the structure and quality of data as it flows through each layer of the architecture (from Bronze ⇒ Silver ⇒ Gold layer tables).

Medallion Architecture Use Case: wetter.com

- Germany weather portal with up to 20 million monthly unique users
- Monetize data by selling data-products to business customers
- Decode the interrelation between weather, consumer behavior and many other factors used by clients in retail, FMCG, e-commerce, tourism, food and advertising
- Switch from multiple steps Spark/S3 data pipeline to Data Lakehouse architecture



https://www.databricks.com/blog/2020/11/13/how-to-evaluate-data-pipelines-for-cost-to-performance.html

Data Lakehouse and Medallion Architecture

- Data Lakehouse architecture focuses on simplicity by processing data in the data store from bronze to silver to gold
- Built-in data governance including ACID transaction control, schema enforcement, etc.
  - Recording all changes and commits in transaction log ensures data integrity and reliability
  - Time-travel: Each write creates a new data version, old version can be view and revert (rollback)
- Key Principle: single source of truth as there is only one path